

Limitations

In this work, we mainly focus on the zero-shot setting to investigate how large multimodal models face reasoning challenges in novel situations. However, previous works have shown that prompting with demonstrations (Brown et al., 2020) may improve the models ability to adapt to new tasks. Hence, we also include experiments in the few-shot setting in Section 5.3, which showed inconsistent benefits, and we aim to explore this area in the future works.

References

- Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C. Lawrence Zitnick, and Devi Parikh. 2015. VQA: Visual Question Answering. In *International Conference on Computer Vision (ICCV)*.
- Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2024. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond.
- Tom Brown, Benjamin Mann, Nick Ryder, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.
- Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, John A. Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuan-Fang Li, Scott M. Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *ArXiv*, abs/2303.12712.
- Susan Carey. 2000. The origin of concepts. *Journal of Cognition and Development*, 1:37 – 41.
- Raymond Bernard Cattell. 1963. Theory of fluid and crystallized intelligence: A critical experiment. *Journal of Educational Psychology*, 54:1–22.
- Charles Cole. 1996. Fluid concepts and creative analogies: Computer models of the fundamental mechanisms of thought. *Journal of the Association for Information Science and Technology*, 47:403–404.
- Bosheng Ding, Chengwei Qin, Linlin Liu, Lidong Bing, Shafiq R. Joty, and Boyang Albert Li. 2022. Is gpt-3 a good data annotator? In *Annual Meeting of the Association for Computational Linguistics*.
- Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C Lawrence Zitnick, and Ross Girshick. 2017. Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In *CVPR*.
- Takeshi Kojima, Shixiang (Shane) Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213. Curran Associates, Inc.
- Brenden M. Lake, Tal Linzen, and Marco Baroni. 2019. Human few-shot learning of compositional instructions. In *Annual Meeting of the Cognitive Science Society*.
- Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. 2023. Improved baselines with visual instruction tuning.
- Pan Lu, Swaroop Mishra, Tony Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. 2022. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *Advances in Neural Information Processing Systems*.
- Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. 2019. Ok-vqa: A visual question answering benchmark requiring external knowledge. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Mark P. Mattson. 2014. Superior pattern processing is the essence of the evolved human brain. *Frontiers in Neuroscience*, 8.
- Arseny Moskvichev, Victor Vikram Odouard, and Melanie Mitchell. 2023. The conceptarc benchmark: Evaluating understanding and generalization in the arc domain. *ArXiv*, abs/2305.07141.
- OpenAI. 2023. Gpt-4v(ision) system card.
- Jean Piaget. 1976. Piaget’s theory.
- Linlu Qiu, Liwei Jiang, Ximing Lu, Melanie Sclar, Valentina Pyatkin, Chandra Bhagavatula, Bailin Wang, Yoon Kim, Yejin Choi, Nouha Dziri, and Xiang Ren. 2024. Phenomenal yet puzzling: Testing inductive reasoning capabilities of language models with hypothesis refinement. In *The Twelfth International Conference on Learning Representations*.
- Pratyusha Sharma, Tamar Rott Shaham, Manel Baradad, Stephanie Fu, Adrian Rodriguez-Munoz, Shivam Duggal, Phillip Isola, and Antonio Torralba. 2024. A vision check-up for language models. *ArXiv*, abs/2401.01862.
- Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2023. Cognitive architectures for language agents. *ArXiv*, abs/2309.02427.

- Joshua B. Tenenbaum. 2018. [Building machines that learn and think like people](#). In *Adaptive Agents and Multi-Agent Systems*.
- Shengbang Tong, Zhuang Liu, Yuexiang Zhai, Yi Ma, Yann LeCun, and Saining Xie. 2024. [Eyes wide shut? exploring the visual shortcomings of multimodal llms](#). *ArXiv*, abs/2401.06209.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. [Llama: Open and efficient foundation language models](#).
- Hugo Touvron, Louis Martin, Kevin R. Stone, et al. 2023b. [Llama 2: Open foundation and fine-tuned chat models](#). *ArXiv*, abs/2307.09288.
- Taylor W. Webb, Keith J. Holyoak, and Hongjing Lu. 2022. [Emergent analogical reasoning in large language models](#). *Nature Human Behaviour*, 7:1526 – 1541.
- Zhengyuan Yang, Linjie Li, Kevin Lin, Jianfeng Wang, Chung-Ching Lin, Zicheng Liu, and Lijuan Wang. 2023. [The dawn of lmms: Preliminary explorations with gpt-4v\(ision\)](#). *ArXiv*, abs/2309.17421.
- Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhui Chen. 2023. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *arXiv preprint arXiv:2311.16502*.
- Chi Zhang, Feng Gao, Baoxiong Jia, Yixin Zhu, and Song-Chun Zhu. 2019. Raven: A dataset for relational and analogical visual reasoning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Puzzle Category	Multimodal Templates	Test Instances
Numbers	2	200
Colors	2	200
Shapes	2	200
Size	2	200
Numbers & Shapes	2	200
Numbers & Colors	2	200
Numbers & Size	2	200
Shapes & Colors	2	200
Shapes & Size	2	200
Colors & Size	2	200
Total	20	2000

Table 3: Dataset statistics of PUZZLEVQA.

A Appendix

A.1 Multiple-Choice Format Details

To generate multiple choice options for numeric puzzles, we use heuristics based on the range of the number. For example, if the number is less than 10, then we sample from the range 1 to 9. If the number is less than 100, we sample from the range 1 to 99, and so on. For discrete option choices, we sample from the possible objects in the image, such as the list of predefined colors or sizes or shapes.

A.2 Dataset Details

We report the dataset statistics of PUZZLEVQA in Table 3.

A.3 Prompt Examples

We show examples of the textual prompts in Figure 7. Note that the prompt examples correspond to the image and puzzle in Figure 1. We use a consistent prompt format across all abstract puzzles.

A.4 Code Implementation Example

```
import math
import random
from typing import List, Tuple, Dict

from PIL import Image, ImageDraw, ImageFont
from pydantic import BaseModel

class ColorHexagonPattern(BaseModel):
    colors: Dict[str, str] = dict(
        blue="#6fa8dc",
        green="#93c47d",
        yellow="#ffd966",
        red="#e06666",
        purple="#8e7cc3",
        orange="#f6b26b",
    )
    image_size: int = 512
    scale_factor: int = 4
    path_font: str = "fonts/OpenSans-Medium.ttf"

    @staticmethod
```

```
def get_centroid(points: List[Tuple[float, float]]) -> Tuple[float, float]:
    x = sum(p[0] for p in points) / len(points)
    y = sum(p[1] for p in points) / len(points)
    return x, y

def sample_colors(self) -> Tuple[List[str], List[str]]:
    while True:
        names = random.sample(list(self.colors), k=3)
        if "orange" in names and "yellow" in names:
            continue # Hard to distinguish
        names = names + names
        colors = [self.colors[n] for n in names]
        return names, colors

def make_sample(self):
    # Set the size of the image
    size = self.image_size * self.scale_factor
    image = Image.new("RGB", size=(size, size), color="white")
    draw = ImageDraw.Draw(image)
    center = size // 2

    # Hexagon properties
    length = size // 3 # Length of a side of the hexagon and triangles
    triangle_height = math.sqrt(3) / 2 * length

    # The vertices of the hexagon
    hexagon = [
        (center + length / 2, center - triangle_height),
        (center - length / 2, center - triangle_height),
        (center - length, center),
        (center - length / 2, center + triangle_height),
        (center + length / 2, center + triangle_height),
        (center + length, center),
    ]

    # Colors for the triangles
    names, colors = self.sample_colors()
    i_answer = random.randint(0, len(colors) - 1)
    answer = names[i_answer]
    colors[i_answer] = "#eeeeee" # Grey

    # Draw the hexagon made of six triangles
    for i in range(6):
        # Coordinates of the triangle vertices
        triangle = [hexagon[i], hexagon[(i + 1) % 6], (
            center, center)]
        # Draw the triangle
        draw.polygon(triangle, fill=colors[i])
        # Draw the outline with custom width
        points = [hexagon[i], hexagon[(i + 1) % 6], (
            center, center), hexagon[i]]
        draw.line(points, fill="black", width=self.scale_factor * 4)

    # Draw "?" on the missing answer part
    if i == i_answer:
        draw.text(self.get_centroid(triangle),
            text="?",
            font=ImageFont.truetype(self.path_font,
                size=size // 10),
            anchor="mm",
            fill="black",
        )

    names[i_answer] = "?"
    instances = sorted(set(n for n in names if n not in
        [answer, "?"]))
    image = image.resize((self.image_size, self.image_size), Image.LANCZOS)
    return (
        dict(
            question="What is the missing color of the
                part denoted with a question mark?",
            answer=answer,
            options=sample_options(answer, options=list(
                self.colors), k=4),
            caption=f"There is a hexagon split into six
                parts with the colors {names} in an
                anti-clockwise order.",
            explanation=f"We observe that a {instances
                [0]} part is opposite another {
                instances[0]} part, and a {
                instances[1]} part is opposite
                another {instances[1]} part. Thus,
                the pattern is that the colors in
                opposite parts are the same.",
            deduction=f"Based on the pattern that
                spatially opposite parts have the
                same color, the missing color of
                the part which is opposite a {
                answer} part should be {answer}.",
        ),
        image,
    )
```